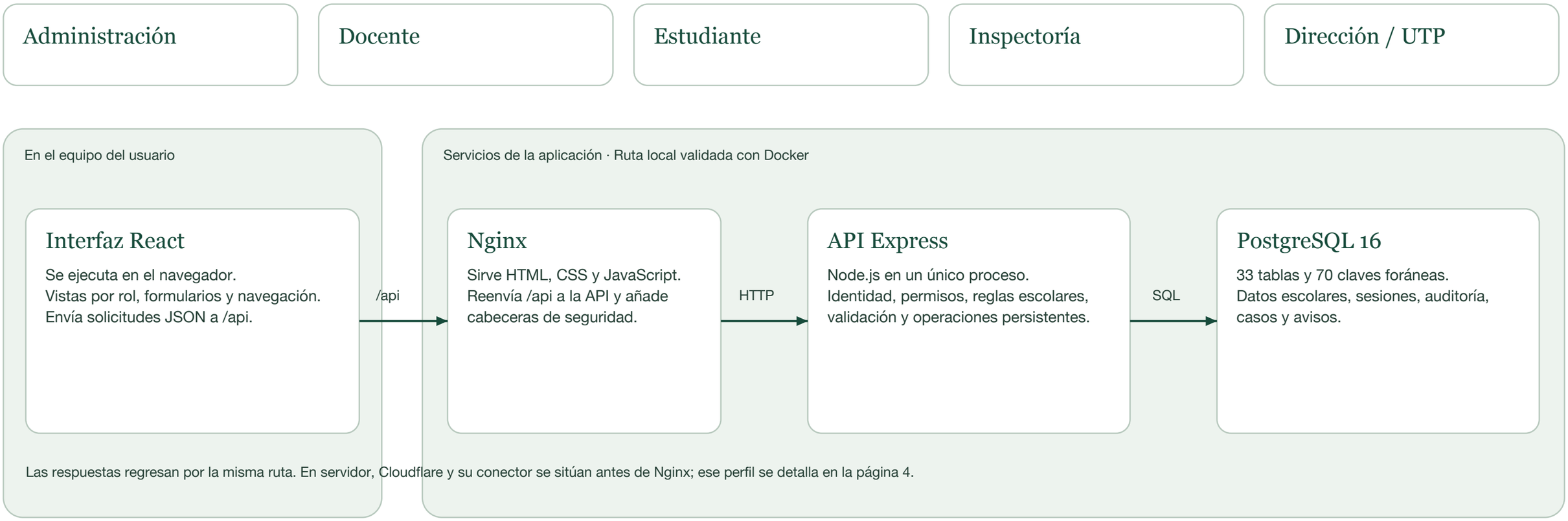
01 / Vista general del sistema

Aplicación web para la gestión académica y el seguimiento de estudiantes de colegios y liceos.



Decisión arquitectónica

Monolito modular con separación entre presentación, API y persistencia. Los módulos funcionales comparten proceso, conexión PostgreSQL y controles de acceso. Esta organización permite transacciones coherentes y un despliegue Docker manejable por el equipo.

Herramientas de trabajo del proyecto, fuera de las solicitudes de los usuarios

GitHub · Código, documentos y revisión

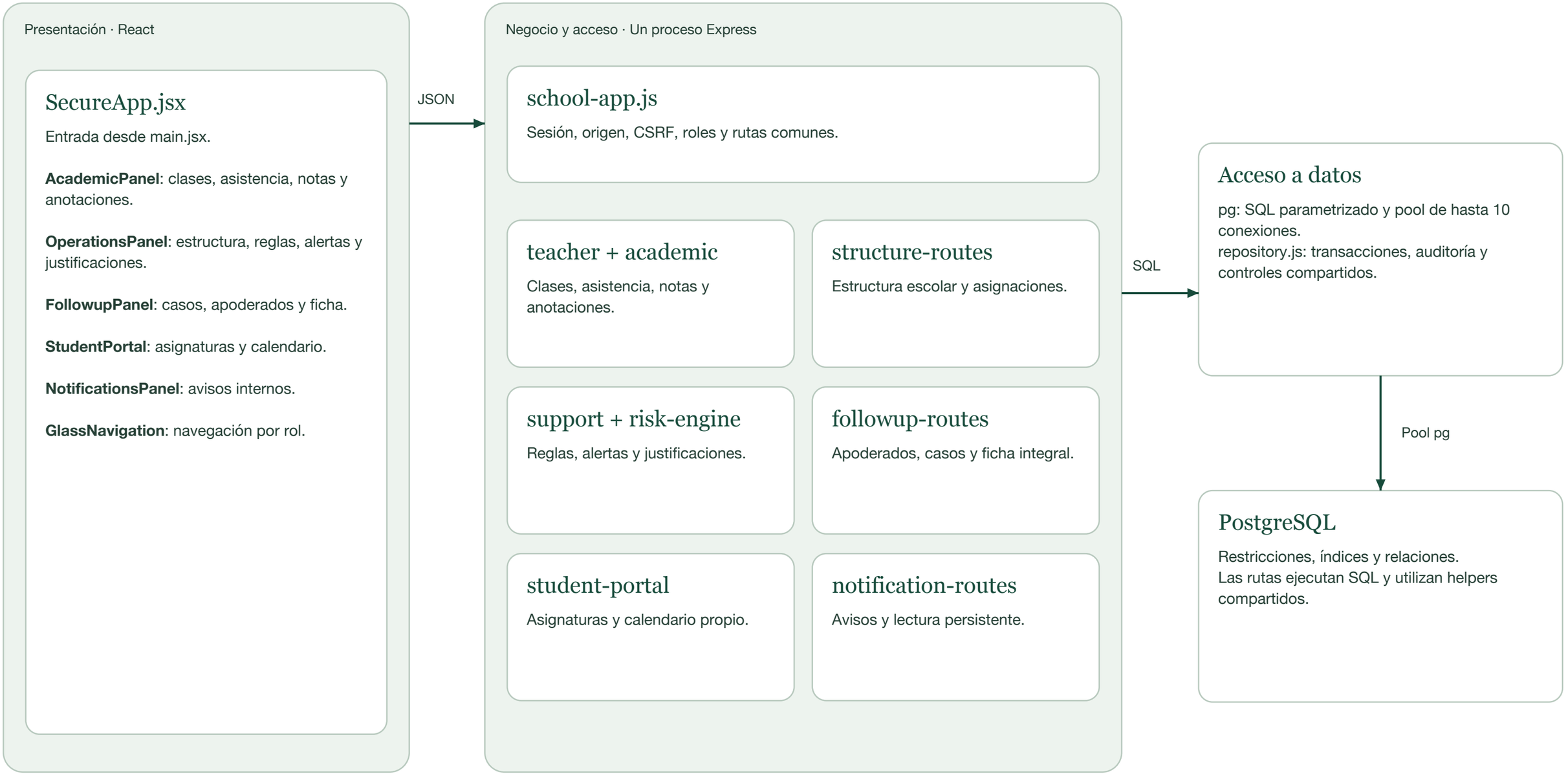
Jira · Historias, tareas y sprints

Docker Compose · Ejecución y pruebas

Fuentes: main.jsx, SecureApp.jsx, school-app.js, server.js, nginx.conf y compose.yaml.

02 / Componentes y módulos

La interfaz consume la API; la autorización y las reglas se resuelven en el servidor.



Los módulos anteriores son archivos del mismo servicio. Las validaciones de rol y curso se ejecutan aunque la petición no provenga de la interfaz. Los avisos se insertan dentro de la transacción del caso; su consistencia no depende de una cola externa.

Fuentes: SecureApp.jsx, paneles React, school-app.js, *-routes.js, risk-engine.js y repository.js.

03 / Seguridad de una solicitud

Sesiones persistentes y comprobación de permisos en cada operación protegida.

A. Inicio de sesión

Credenciales

Correo y contraseña en el navegador.

Validación

Origen exacto, límite de intentos y scrypt.

Persistencia

Hash del token, usuario y expiración en PostgreSQL.

Sesión del navegador

Cookie HttpOnly y token CSRF devueltos al cliente.

B. Operación protegida: por ejemplo, modificar un caso

Origen y formato

Origen permitido, JSON y cuerpo de hasta 32 kB.

Identidad vigente

Cookie hash sesión no expirada ni revocada.

Autorización

Rol actual, curso o matrícula y token CSRF al escribir.

Operación SQL

Validar datos y revisión; confirmar la transacción.

Sesión y contraseña

Cookie Secure en producción, SameSite=Strict y ruta /. Duración configurada: 8 horas, con inactividad máxima de 30 minutos. Cambiar contraseña revoca sesiones. La base conserva hashes del token y de la contraseña.

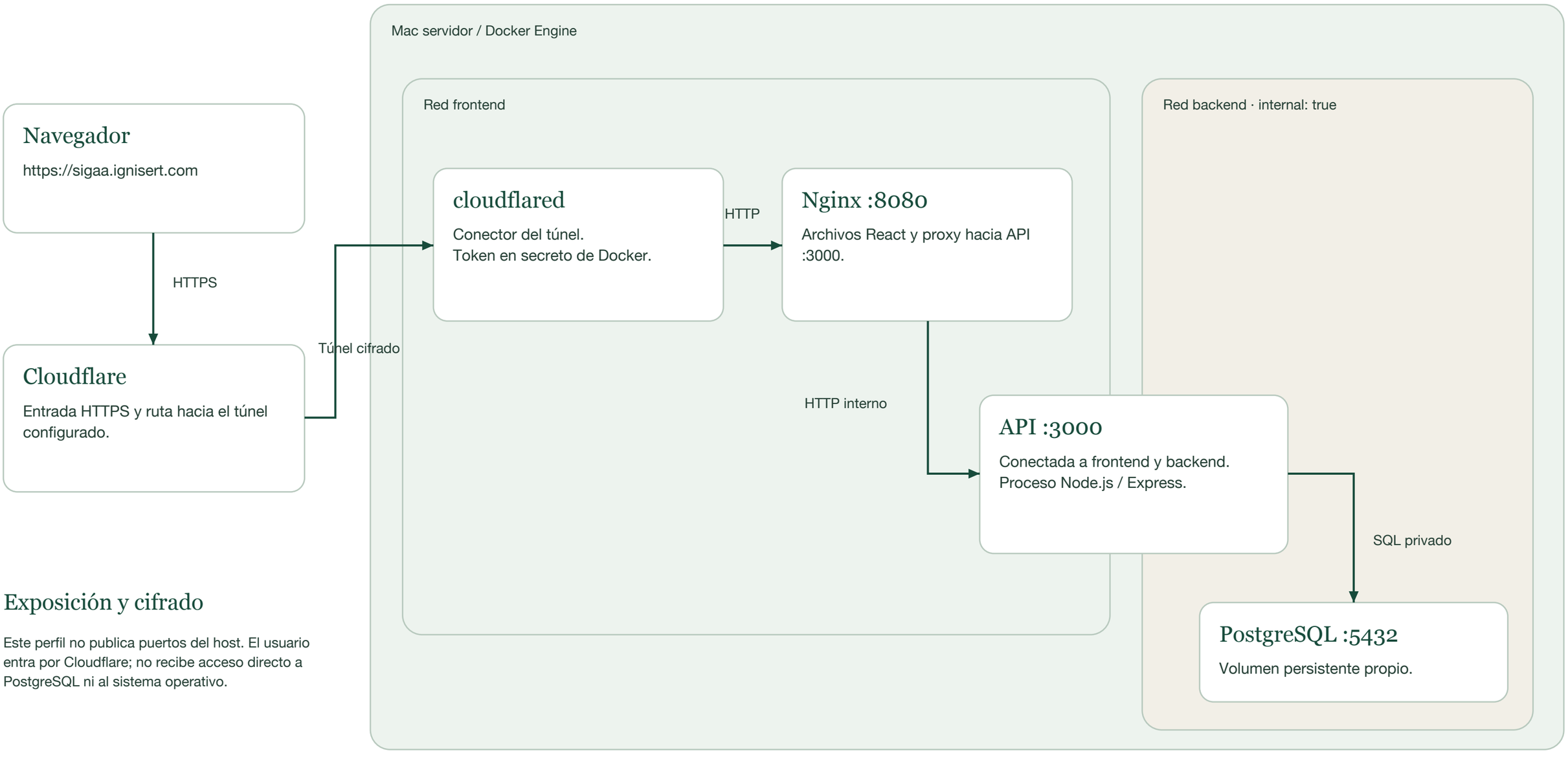
Respuesta ante una condición inválida

La petición se rechaza antes de modificar el recurso: 401 por sesión inválida; 403/404 según el recurso fuera del alcance; 409 ante una revisión desactualizada; 429 al superar límites. El menú visible no determina el permiso efectivo.

Fuentes: school-app.js, security.js, origins.js, repository.js y migración 007.

04 / Despliegue de servidor configurado

Perfil compose.server.yaml · Arquitectura preparada para HTTPS, no auditoría del servidor público actual.



Exposición y cifrado

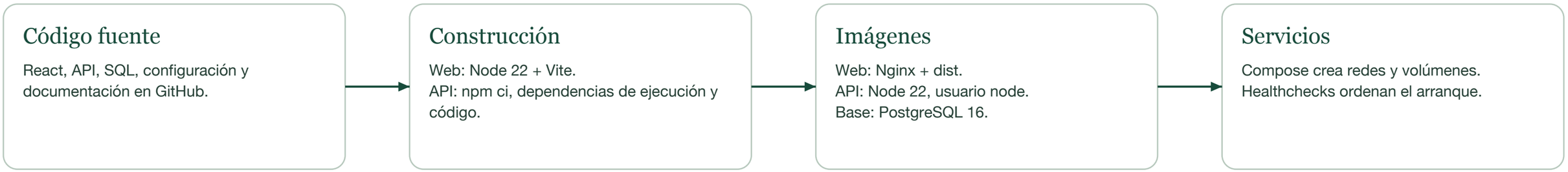
Este perfil no publica puertos del host. El usuario entra por Cloudflare; no recibe acceso directo a PostgreSQL ni al sistema operativo.

HTTPS termina en Cloudflare y el enlace al conector va cifrado. Los tramos entre contenedores usan HTTP y SQL privados sin TLS explícito en esta configuración. El aislamiento de red y el cifrado son controles distintos. Las flechas representan solicitudes; el conector establece el túnel saliente.

Fuentes: compose.server.yaml, nginx.conf, Dockerfile de API/web y origins.js.

05 / Construcción y configuraciones Docker

Separación entre compilación, ejecución local y perfiles de servidor disponibles en el repositorio.



Perfiles existentes

Perfil	Conexión y aislamiento	Alcance del documento
compose.yaml	Base propia sin puerto al host. Web 127.0.0.1:8088 y API 127.0.0.1:3000 por defecto; red bridge compartida.	Instalación y reinicio de 0.9.1 verificados en un proyecto Docker aislado.
compose.server.yaml	Web/conector en frontend; API en ambas redes; base en backend interno. Túnel mediante token secreto, sin ports del host.	Configuración preparada; esta entrega no la aplica ni acredita despliegue 0.9.1.
compose.hosted.yaml	API y web del despliegue separado. Base externa en sigaa_database_private; conector de URL temporal hacia web.	La base y la configuración del túnel permanente se administran aparte. Revisar antes de actualizar producción.

Desarrollo y actualización

Vite se usa para desarrollo y compilación; en la imagen web los archivos se sirven con Nginx. Nginx resuelve el servicio api por DNS de Docker con caché de cinco segundos para recuperarse de cambios de IP.

Una instalación vacía aplica 001-010. Una base escolar existente usa el migrador aditivo 007-010, tras respaldo y ensayo de restauración. server.js comprueba las versiones de esquema antes de escuchar. La configuración hosted exige preparar la migración con su entorno y permisos.

Fuentes: Dockerfile de web/API, compose.yaml, compose.hosted.yaml, compose.server.yaml y vite.config.js.

06 / Flujo persistente de casos y avisos

El cambio del caso, su historial, la auditoría y sus avisos se confirman como una sola operación.

Personal autorizado

Abre o modifica un caso desde la interfaz escolar.

API de seguimiento

Comprueba rol, curso, responsable y expectedRevision.

Bloqueo y transacción

Bloquea el registro y rechaza ediciones desactualizadas.

Escrituras atómicas

Misma transacción PostgreSQL

caso_seguimiento

Estado, responsable, plan y revisión.

caso_actividad

Historial de cambios e intervenciones.

evento_auditoria

Actor y cambios registrados.

notificacion_interna

Referencia al evento y sus destinatarios.

Resultado

COMMIT confirma el conjunto.
ROLLBACK revierte también el aviso.
Una clave única evita duplicados.

Bandeja de avisos

Consulta manual, filtros y paginación; lectura persistente.

Permiso comprobado de nuevo

Destinatario, rol y acceso actual al curso. “Ver caso” abre el recurso autorizado.

Datos del aviso

Creador/responsable activos, excluyendo al actor. Se guarda la referencia, no una copia de las notas del caso.

Alcance implementado: avisos internos de casos. Correo, push, mensajería externa y cuentas de apoderados no forman parte de este flujo.

Fuentes: followup-routes.js, notification-routes.js, repository.js y migraciones 009-010.

07 / Trazabilidad y estado de verificación

Arquitectura del candidato 0.9.1; fuentes versionadas y límites de la evidencia disponible.

Componente	Tecnología declarada	Responsabilidad
Interfaz	React 19.2.8 + Vite 8.2.4	Presentación, estado de interfaz y formularios por rol.
Entrada web	Nginx 1.27-alpine	Contenido estático, proxy /api y cabeceras de seguridad.
Servidor de aplicación	Node 22-alpine + Express 5.2.1	Sesiones, permisos, validación, reglas escolares y rutas de negocio.
Acceso SQL / datos	pg 8.23.0 + PostgreSQL 16-alpine	SQL parametrizado, transacciones, índices, restricciones y persistencia.
Infraestructura	Docker Compose + cloudflared	Contenedores, redes, volúmenes y túnel según el perfil elegido.

Evidencia ya registrada

6 pruebas unitarias y 66 de integración aprobadas con Docker/PostgreSQL 16; 27 recorridos Chromium; build web; instalación nueva y conservación de cuentas/sesiones tras reiniciar base y API. La evidencia está en output/evidence/sprint-5-completion/. No se repitieron esas pruebas para dibujar este PDF.

Alcance pendiente

Aceptación del Product Owner, integración a main y despliegue del incremento. Sigue pendiente el ensayo de HTTPS desde los equipos del grupo y la restauración de respaldo. Los diagramas no certifican una auditoría en vivo del servidor ni el cierre administrativo de los sprints.

Fuentes para reproducir y revisar

Código base: d24d60e. El manifiesto architecture-software-0.9.1.json registra 35 archivos y sus huellas SHA-256. Incluye rutas, paneles, configuración de red, migraciones y evidencia previa. El esquema de datos completo se consulta en el PDF de base de datos del mismo directorio.

Fuentes: package.json, Dockerfiles, validation.json y database-schema-0.9.1.json.